

| Quality Assurance in SW-Development

Assignment 2: Property-Based Testing with ScalaCheck

Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security

What is Property-Based Testing?

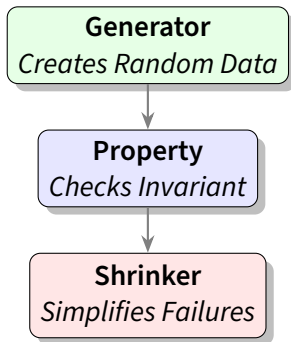
The Concept

Instead of writing specific examples:

```
assert(add(2,2) == 4)
```

We define **general properties** (invariants):

*"For any integers a and b ,
 $add(a, b)$ should equal $add(b, a)$ "*



Anatomy of a ScalaCheck Test

```
1 import org.scalacheck.Prop.forAll
2
3 // A simple property: String concatenation length
4 val prop_ConcatString = forAll { (s1: String, s2: String) =>
5     (s1 ++ s2).length == s1.length + s2.length
6 }
7
8 // Running the check
9 prop_ConcatString.check
10 // > + OK, passed 100 tests.
11
```

⚙️ How it works:

- `forAll` takes a function.
- ScalaCheck automatically generates 100 random values for `s1` and `s2`.

⚠️ Key takeaway:

- You describe the **rules**.
- The framework finds the **edge cases**.

Generators (Gen)

Sometimes the default random data isn't enough. We need structure.

```
1 import org.scalacheck.Gen
2
3 // Generate a number between 0 and 100
4 val myIntGen = Gen.choose(0, 100)
5
6 // Generate a list of boolean values
7 val myBoolList = Gen.listOf(Gen.oneOf(true, false))
8
9 // Generate specific strings
10 val titleGen = Gen.oneOf("Mr.", "Mrs.", "Dr.")
11
```

💡 **Assignment Hint:** You will need custom Generators for the Message objects!

Implication (Preconditions)

What if a property only holds for *some* inputs?

```
1 import org.scalacheck.Prop.propBoolean // Enable ==> syntax
2
3 val prop_Sqrt = forAll { (n: Int) =>
4     (n >= 0) ==> { // IMPLICATION OPERATOR (Alternative: whenever)
5         Math.sqrt(n * n) == n
6     }
7 }
8
```

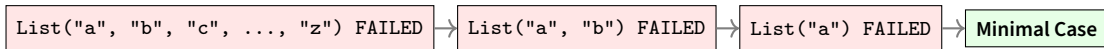
⚠ The Danger of ==>

If the condition is hard to satisfy (e.g., $n == 0$), ScalaCheck will discard too many tests and eventually **give up** (Exhausted).

- **Better:** Use a Generator that only produces valid data (e.g., `Gen.posNum[Int]`).

Shrinking (The Magic)

When a test fails, ScalaCheck doesn't just give you the random input. It tries to find the **smallest possible failure**.



Why is this useful?

- Debugging a list of 1000 items is hard.
- Debugging a list of 1 item is easy.
- ScalaCheck does this automatically!

Assignment 2: The Goal



</> The Simple Part

Simple Java/Scala Functions

- You get a few basic functions.
- Write ScalaCheck properties to verify them.
- Think about: Commutativity, Identity, Associativity.

✉ The Hard Part

The Message Board

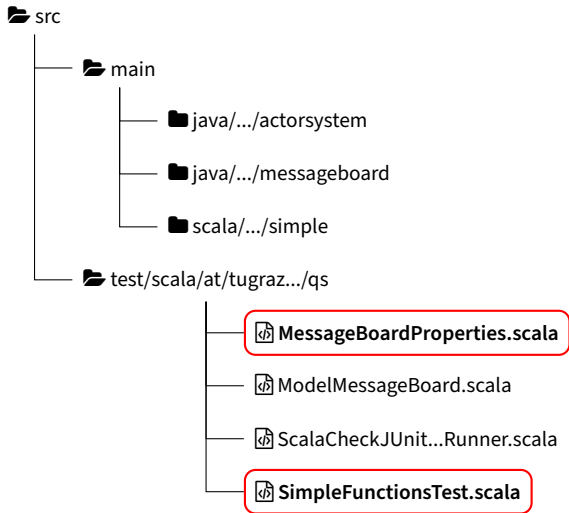
- You are given the Message Board from A1.
- **Goal:** Write properties to verify its behavior.
- **Challenge:** You need to generate random Message objects.

Assignment 2: The Goal

i Pro Tip:

Don't test the *implementation*. Test the *behavior*.
"If I publish a message, does it appear in the list?"

Project Structure



Project Structure

Your Tasks

You need to modify these two files:

1 **SimpleFunctionsTest.scala:**

- Write properties for the simple math/list functions provided in `src/main/scala`.

2 **MessageBoardProperties.scala:**

- Write Generators for Messages.
- Write Properties to test the Java MessageBoard implementation.

Hints for the Second Partial Exam

Pen & Paper

Be ready to write code on paper!

- **Small Properties:** Given a function (e.g., `reverseList`), write a property to test it.
- **Generators:** Write a Generator for a simple class (e.g., a `Person` with age 0-100).

Theory Concepts

Understand the definitions:

- **Shrinking:** What is it? Why do we need it? (Finding the minimal counter-example).
- **Generators:** How do we combine small generators into big ones?

 **Practice:** Try writing the `Gen.oneOf` logic on a piece of paper!

Questions?

Live-Demo :)

Any final Questions?

| Quality Assurance in SW-Development

Assignment 2: Property-Based Testing with ScalaCheck

Sebastian David Albu

Summer Semester 2026

Institute of Software Engineering and Artificial Intelligence

LaTeX Template provided by the Institute of Information Security